TUGAS PRAKTIKUM 1

SISTEM OPERASI

MUHAMMAD MAULANA NUR SULAIMAN

2410651033

UNIVERSITAS MUHAMMADIYAH JEMBER

FAKULTAS TEKNIK

PROGRAM STUDI TEKNIK INFORMATIKA

2025

DAFTAR ISI

Contents

Tugas 1: Process Manager

Konsep dan struktur yang ditunjukkan

1. fork() – membuat proses anak (child process)
2. pipe() – komunikasi antar process (ipc) satu arah
3. shmget(), shmat(), (Shared Memory) – komunikasi antar memori bersama
4. waitpid() – parent menunggu setiap child selesai agar tidak terjadi zombie proses
5. signal handling (sigint) – menangani sinyal Ctrl+C agar program berhenti dengan rapi
6. gettimeofday() – mengukur waktu eksekusi tiap proses anak

Alur kerja program

1. Program meminta digit terahir
    - Jika ganjil, Child 2 akan berkomunikasi dengan parent melalui shared memory
2. Parent meminta tiga input
    - Angka untuk faktorial (Child 1)
    - Batas untuk bilangan prima (Child 2)
    - String untuk dibalik (Child 3)
3. Parent membuat 3 pipe dan 3 child (melalui fork())
    - Child 1: menghitung factorial – kirim hasil lewat pipe
    - Child 2: mencari bilangan prima
    - Child 3: membalik string input – kirim hasil lewat pipe
4. Parent menunggu tiap child selesaai (waitpid), mencatat waktu eksekusi, lalu:
    - Membaca hasil dari pipe atau shared memory
    - Menampilkan hasil dan durasi eksekusinya
5. Setelah semua selesai, parent menghapus shared memory (jika ada) dan keluar dengan pesan akhir

```c
#define _XOPEN_SOURCE 700
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/time.h>   // gettimeofday
#include <sys/wait.h>   // waitpid, macros
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>    // shmget, shmat
#include <signal.h>
#include <errno.h>

#define CHILD_COUNT 3
#define SHM_SIZE 4096

pid_t child_pids[CHILD_COUNT];
int pipes_arr[CHILD_COUNT][2];
int shmid = -1;
int use_shm_for_child = -1; // which child uses shared memory (-1 = none)
volatile sig_atomic_t sigint_received = 0;
int nim_last_digit = 0;

void sigint_handler(int signo) {
    // Signal handler for SIGINT in parent (to gracefully terminate children).
    sigint_received = 1;
    // We DO NOT use non-async-signal-safe calls here except kill (which is safe).
    for (int i = 0; i < CHILD_COUNT; ++i) {
        if (child_pids[i] > 0) {
            kill(child_pids[i], SIGTERM); // request child termination
        }
    }
}

/* Utility: write full buffer to fd (handles short writes) */
ssize_t write_full(int fd, const void *buf, size_t count) {
    const char *p = buf;
    size_t left = count;
    while (left > 0) {
        ssize_t w = write(fd, p, left);
        if (w < 0) {
```

```
            if (errno == EINTR) continue;
            return -1;
        }
        left -= w;
        p += w;
    }
    return count;
}

/* Utility: read full until EOF into dynamically allocated buffer.
   Caller frees returned pointer. */
char *read_all_fd(int fd) {
    size_t cap = 1024;
    size_t len = 0;
    char *buf = malloc(cap);
    if (!buf) return NULL;
    while (1) {
        ssize_t r = read(fd, buf + len, cap - len);
        if (r < 0) {
            if (errno == EINTR) continue;
            free(buf);
            return NULL;
        } else if (r == 0) {
            break;
        } else {
            len += r;
            if (len == cap) {
                cap *= 2;
                char *t = realloc(buf, cap);
                if (!t) { free(buf); return NULL; }
                buf = t;
            }
        }
    }
    buf[len] = '\0';
    return buf;
}
```

```c
/* Factorial (iterative, returns as unsigned long long).
   For simplicity, we won't handle overflow checks here beyond small inputs. */
unsigned long long factorial(int n) {
    unsigned long long res = 1;
    for (int i = 2; i <= n; ++i) res *= i;
    return res;
}

/* Simple prime sieve (Sieve of Eratosthenes) returning a string of primes separated by space.
   Caller must free returned string. */
char *primes_up_to(int limit) {
    if (limit < 2) {
        char *res = strdup("");
        return res;
    }
    char *isprime = calloc(limit + 1, 1);
    for (int i = 2; i <= limit; ++i) isprime[i] = 1;
    for (int p = 2; p * p <= limit; ++p) {
        if (isprime[p]) {
            for (int k = p * p; k <= limit; k += p) isprime[k] = 0;
        }
    }
    // Build string
    size_t cap = 256;
    char *out = malloc(cap);
    size_t used = 0;
    out[0] = '\0';
    for (int i = 2; i <= limit; ++i) {
        if (isprime[i]) {
            char tmp[64];
            int written = snprintf(tmp, sizeof(tmp), "%d ", i);
            if (used + written + 1 > cap) {
                cap *= 2;
                out = realloc(out, cap);
            }
            strcat(out, tmp);
            used += written;
        }
    }
```

```c
    free(isprime);
    return out;
}

/* Reverse string (returns newly allocated string) */
char *reverse_str(const char *s) {
    size_t n = strlen(s);
    char *r = malloc(n + 1);
    for (size_t i = 0; i < n; ++i) r[i] = s[n - 1 - i];
    r[n] = '\0';
    return r;
}

int main() {
    printf("=== Simple Process Manager (fork, pipe, shm, waitpid) ===\n");

    // 1) Read NIM last digit to pick behavior
    printf("Masukkan digit terakhir NIM Anda (0-9): ");
    fflush(stdout);
    if (scanf("%d", &nim_last_digit) != 1) {
        fprintf(stderr, "Input invalid. Exiting.\n");
        exit(EXIT_FAILURE);
    }
    while (getchar() != '\n'); // flush rest of line

    // Decide behavior: if odd -> use shared memory for one child; if even -> install SIGINT handler
    if (nim_last_digit % 2 == 1) {
        use_shm_for_child = 1; // we'll use shared memory for child index 1 (Child 2)
        printf("NIM akhir GANJIL -> Child 2 akan menggunakan shared memory untuk komunikasi.\n");
    } else {
        use_shm_for_child = -1;
        printf("NIM akhir GENAP -> Parent memasang SIGINT handler untuk graceful termination.\n");
        // install SIGINT handler
        struct sigaction sa;
        sa.sa_handler = sigint_handler;
        sigemptyset(&sa.sa_mask);
        sa.sa_flags = 0;
        if (sigaction(SIGINT, &sa, NULL) == -1) {
            perror("sigaction");
            // not fatal; continue
        }
```

```
}

// Prepare input values for children (ask once)
int fact_n = 0;
int prime_limit = 0;
char input_str[1024];

printf("Input untuk Child 1 (faktorial): masukkan angka non-negatif (mis. 5): ");
fflush(stdout);
if (scanf("%d", &fact_n) != 1) { fprintf(stderr, "Invalid input\n"); exit(EXIT_FAILURE); }
while (getchar() != '\n');

printf("Input untuk Child 2 (prima): masukkan batas (mis. 30): ");
fflush(stdout);
if (scanf("%d", &prime_limit) != 1) { fprintf(stderr, "Invalid input\n"); exit(EXIT_FAILURE); }
while (getchar() != '\n');

printf("Input untuk Child 3 (reverse string): masukkan string (max 1000 chars): ");
fflush(stdout);
if (!fgets(input_str, sizeof(input_str), stdin)) { fprintf(stderr, "Invalid input\n"); exit(EXIT_FAILURE); }
// strip newline
size_t L = strlen(input_str);
if (L && input_str[L-1] == '\n') input_str[L-1] = '\0';

// Create pipes for each child
for (int i = 0; i < CHILD_COUNT; ++i) {
    // pipe() digunakan untuk membuat channel IPC unidirectional antara processes
    if (pipe(pipes_arr[i]) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }
}
```

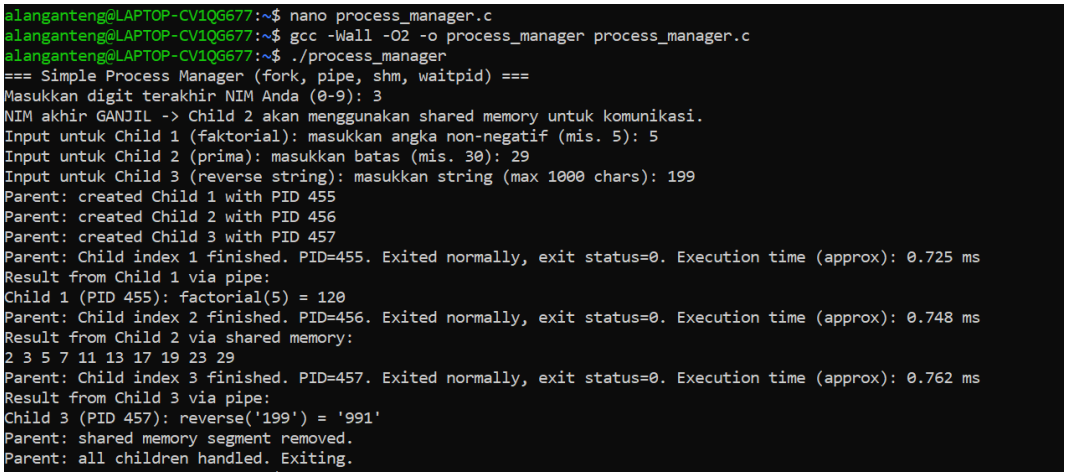
```
// If shared memory used, create shmid now (parent will remove later)
if (use_shm_for_child != -1) {
    // shmget() dibuat dengan IPC_PRIVATE sehingga IPC key khusus.
    // ukurannya SHM_SIZE, permission 0666.
    shmid = shmget(IPC_PRIVATE, SHM_SIZE, IPC_CREAT | 0666);
    if (shmid == -1) {
        perror("shmget");
        // fallback: disable shm and use pipe instead
        fprintf(stderr, "Gagal membuat shared memory; fallback ke pipe untuk semua child.\n");
        use_shm_for_child = -1;
    }
}

// Keep timestamps for each child measured by parent: start when fork returns child pid
struct timeval start_times[CHILD_COUNT];
struct timeval end_times[CHILD_COUNT];
for (int i = 0; i < CHILD_COUNT; ++i) { start_times[i].tv_sec = start_times[i].tv_usec = 0; end_times[i] = start_times[i]; }

// Fork CHILD_COUNT children
for (int i = 0; i < CHILD_COUNT; ++i) {
    pid_t pid = fork(); // fork() digunakan untuk membuat process baru (child) sebagai copy dari parent
    if (pid < 0) {
        perror("fork");
        // try to terminate already created children
        for (int j = 0; j < i; ++j) if (child_pids[j] > 0) kill(child_pids[j], SIGTERM);
        exit(EXIT_FAILURE);
    } else if (pid == 0) {
        // ========== Child process code ==========
        // Close unused pipe ends: child only writes to parent (write end), so close read end.
        for (int k = 0; k < CHILD_COUNT; ++k) {
            if (k == i) {
                close(pipes_arr[k][0]); // close read end for this child's pipe
                // keep write end open
            } else {
                // close both ends for other pipes not used by this child
                close(pipes_arr[k][0]);
                close(pipes_arr[k][1]);
            }
        }
```

```
// If this child uses shared memory (only applies if configured for this child index),
// it will attach and write result into shared memory instead of pipe.
if (use_shm_for_child == i) {
    // Attach to shared memory: shmat returns pointer
    void *shmaddr = shmat(shmid, NULL, 0);
    if (shmaddr == (void *) -1) {
        // Failed to attach: write an error message to pipe fallback
        char fallback_msg[256];
        snprintf(fallback_msg, sizeof(fallback_msg), "Child %d: Failed shmat: %s\n", i+1, strerror(errno));
        write_full(pipes_arr[i][1], fallback_msg, strlen(fallback_msg));
        close(pipes_arr[i][1]);
        _exit(EXIT_FAILURE);
    }
    // Perform computation for designated child
    if (i == 1) {
        // Child 2: primes -> write to shared memory
        char *primes_s = primes_up_to(prime_limit);
        // Copy into shared memory area (null-terminated)
        strncpy((char *)shmaddr, primes_s, SHM_SIZE - 1);
        ((char *)shmaddr)[SHM_SIZE - 1] = '\0';
        free(primes_s);
        // Detach and exit
        shmdt(shmaddr); // shmdt detaches the shared memory segment in the child
        close(pipes_arr[i][1]); // no pipe use but close write end
        _exit(0);
    } else {
        // Just in case config pointed to other child, fallback
        snprintf((char *)shmaddr, SHM_SIZE, "Child %d (unexpected) wrote to shm.\n", i+1);
        shmdt(shmaddr);
        close(pipes_arr[i][1]);
        _exit(0);
    }
} else {
    // Use pipe-based communication for this child (common case)
    if (i == 0) {
        // Child 1: factorial
        unsigned long long f = factorial(fact_n);
        char out[256];
        int n = snprintf(out, sizeof(out), "Child 1 (PID %d): factorial(%d) = %llu\n", getpid(), fact_n, f);
        write_full(pipes_arr[i][1], out, n);
        close(pipes_arr[i][1]);
```

```
                _exit(0);
            } else if (i == 1) {
                // Child 2: primes (when not using shm)
                char *pr = primes_up_to(prime_limit);
                char header[128];
                int h = snprintf(header, sizeof(header), "Child 2 (PID %d): primes <= %d: ", getpid(), prime_limit);
                write_full(pipes_arr[i][1], header, h);
                write_full(pipes_arr[i][1], pr, strlen(pr));
                write_full(pipes_arr[i][1], "\n", 1);
                free(pr);
                close(pipes_arr[i][1]);
                _exit(0);
            } else if (i == 2) {
                // Child 3: reverse string
                char *rev = reverse_str(input_str);
                char out[2048];
                int n = snprintf(out, sizeof(out), "Child 3 (PID %d): reverse('%s') = '%s'\n", getpid(), input_str, rev);
                write_full(pipes_arr[i][1], out, n);
                free(rev);
                close(pipes_arr[i][1]);
                _exit(0);
            } else {
                // Should not happen
                const char *msg = "Unknown child\n";
                write_full(pipes_arr[i][1], msg, strlen(msg));
                close(pipes_arr[i][1]);
                _exit(EXIT_FAILURE);
            }
        }
    } else {
        // ========== Parent process after fork ==========
        child_pids[i] = pid;
        // Parent closes the write end of the child's pipe (we read from read end).
        close(pipes_arr[i][1]);
        // record start time for this child measurement
        gettimeofday(&start_times[i], NULL);
        printf("Parent: created Child %d with PID %d\n", i+1, pid);
    }
} // end fork loop
```

```c
    // Parent: now wait for children and handle IPC reading
    for (int i = 0; i < CHILD_COUNT; ++i) {
        int status;
        pid_t w;
        // waitpid() digunakan untuk menunggu proses tertentu selesai sehingga parent dapat
        // mengambil status exit dan mencegah zombie process.
        w = waitpid(child_pids[i], &status, 0);
        gettimeofday(&end_times[i], NULL);
        if (w == -1) {
            perror("waitpid");
            continue;
        }

        // Compute elapsed milliseconds
        long sec = end_times[i].tv_sec - start_times[i].tv_sec;
        long usec = end_times[i].tv_usec - start_times[i].tv_usec;
        if (usec < 0) { sec -= 1; usec += 1000000; }
        double elapsed_ms = sec * 1000.0 + usec / 1000.0;

        printf("Parent: Child index %d finished. PID=%d. ", i+1, w);
        if (WIFEXITED(status)) {
            printf("Exited normally, exit status=%d. ", WEXITSTATUS(status));
        } else if (WIFSIGNALED(status)) {
            printf("Terminated by signal %d. ", WTERMSIG(status));
        } else {
            printf("Stopped/Other status.\n");
        }
        printf("Execution time (approx): %.3f ms\n", elapsed_ms);

        // Read result from IPC: either pipe (most cases) or shared memory (for the chosen child)
        if (use_shm_for_child == i) {
            // parent attaches to shared memory to read content
            void *shmaddr = shmat(shmid, NULL, 0);
            if (shmaddr == (void *) -1) {
                fprintf(stderr, "Parent: failed to shmat to read child %d result: %s\n", i+1, strerror(errno));
            } else {
                printf("Result from Child %d via shared memory:\n%s\n", i+1, (char *)shmaddr);
                shmdt(shmaddr); // detach after read
            }
            // Also close the read end of pipe (was left open)
            close(pipes_arr[i][0]);
```

```c
        } else {
            // Read from pipe read-end
            char *res = read_all_fd(pipes_arr[i][0]);
            if (!res) {
                fprintf(stderr, "Parent: failed to read pipe from child %d\n", i+1);
            } else {
                printf("Result from Child %d via pipe:\n%s", i+1, res);
                free(res);
            }
            close(pipes_arr[i][0]);
        }
    }

    // Cleanup shared memory if created
    if (shmid != -1) {
        // Mark segment to be destroyed. shmctl IPC_RMID removes it after detach.
        if (shmctl(shmid, IPC_RMID, NULL) == -1) {
            perror("shmctl(IPC_RMID)");
        } else {
            printf("Parent: shared memory segment removed.\n");
        }
    }

    printf("Parent: all children handled. Exiting.\n");
    return 0;
}
```



Tugas 2: File Monitor

Konsep dan struktur yang ditunjukkan

1. inotify_init(), inotify_add_watch(), dan read() – digunakan untuk memantau perubahan file/direktori
2. open() dan write() – digunakan untuk mencatat aktivitas ke file log.txt
3. signal(SIGTERM, handler) – untuk menangani penghentian program secara aman
4. time() dan localtime – mencatat waktu terjadinya event ke log

Alur kerja program

1. User memasukkan digit NIM terakhir.
    - Jika ganjil - setiap perubahan file disertai pesan “notifikasi email dummy” di terminal.
2. Program menjalankan inotify_init() – Membuat instance inotify untuk memantau direktori yang diberikan di argument (argv[1]).
3. Menambahkan watch dengan inotify_add_watch() – Memonitor event IN_CREATE, IN_DELETE, dan IN_MODIFY.
4. Setiap kali ada perubahan di direktori:
    - Program membaca event inotify_fd menggunakan read()
    - Mencatat ke file log (log.txt) dengan timestamp dan PID process.
5. Menangani SIGTERM (misalnya saat dimatikan dengan kill)
    - Menutup semua file descriptor (inotify_fd, log_fd).
    - Keluar secara aman dari program

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/inotify.h>
#include <sys/types.h>
#include <fcntl.h>
#include <signal.h>
#include <time.h>
#include <string.h>
#include <errno.h>

#define EVENT_BUF_LEN (1024 * (sizeof(struct inotify_event) + 16))
#define LOGFILE "log.txt"

int inotify_fd = -1;
int watch_fd = -1;
int log_fd = -1;
int nim_last_digit = 0;

/* Signal handler: SIGTERM untuk cleanup */
void handle_sigterm(int signo) {
    if (signo == SIGTERM) {
        printf("\nSIGTERM diterima. Menutup inotify dan keluar dengan aman.\n");
        if (watch_fd != -1) close(watch_fd);
        if (inotify_fd != -1) close(inotify_fd);
        if (log_fd != -1) close(log_fd);
        exit(0);
    }
}
```

```c
/* Fungsi menulis log ke file menggunakan system call open(), write() */
void write_log(const char *event_type, const char *filename, pid_t pid) {
    time_t now = time(NULL);
    struct tm *tm_info = localtime(&now);

    char time_buf[64];
    strftime(time_buf, sizeof(time_buf), "%Y-%m-%d %H:%M:%S", tm_info);

    char log_entry[512];
    int len = snprintf(log_entry, sizeof(log_entry), "[%s] [%s] [%s] [PID:%d]\n",
                       time_buf, event_type, filename, pid);

    if (write(log_fd, log_entry, len) == -1) {
        perror("write log");
    }
}

/* Mengecek apakah nama file sesuai filter ekstensi .txt atau .log */
int valid_extension(const char *filename) {
    const char *ext = strrchr(filename, '.');
    if (!ext) return 0;
    return (strcmp(ext, ".txt") == 0 || strcmp(ext, ".log") == 0);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "Usage: %s <directory_to_monitor>\n", argv[0]);
        exit(EXIT_FAILURE);
    }

    printf("Masukkan digit terakhir NIM Anda (0-9): ");
    scanf("%d", &nim_last_digit);

    int filter_mode = (nim_last_digit % 2 == 0) ? 1 : 0;

    if (filter_mode)
        printf("NIM genap -> hanya mencatat file .txt atau .log\n");
    else
        printf("NIM ganjil -> mencetak notifikasi email dummy\n");
```

```c
signal(SIGTERM, handle_sigterm);

/* 1. Membuat inotify instance */
inotify_fd = inotify_init();
if (inotify_fd < 0) {
    perror("inotify_init");
    exit(EXIT_FAILURE);
}

/* 2. Menambahkan watch pada direktori target */
watch_fd = inotify_add_watch(inotify_fd, argv[1],
                             IN_CREATE | IN_MODIFY | IN_DELETE);
if (watch_fd < 0) {
    perror("inotify_add_watch");
    close(inotify_fd);
    exit(EXIT_FAILURE);
}

/* 3. Membuka file log menggunakan system call open() */
log_fd = open(LOGFILE, O_WRONLY | O_CREAT | O_APPEND, 0644);
if (log_fd < 0) {
    perror("open log file");
    close(inotify_fd);
    exit(EXIT_FAILURE);
}

printf("Monitoring direktori: %s\nTekan Ctrl+C atau kirim SIGTERM untuk berhenti.\n", argv[1]);

/* 4. Buffer untuk menampung event dari kernel */
char buffer[EVENT_BUF_LEN];

/* 5. Infinite loop memantau event */
while (1) {
    int length = read(inotify_fd, buffer, EVENT_BUF_LEN);

    if (length < 0) {
        if (errno == EINTR) continue;  // di-interrupt signal
        perror("read");
        break;
    }
```

```c
    /* Struktur inotify_event:
       struct inotify_event {
           int      wd;       // watch descriptor
           uint32_t mask;     // event mask (bitmask event terjadi)
           uint32_t cookie;   // ID untuk event terkait
           uint32_t len;      // panjang field 'name'
           char     name[];   // nama file
       }
    */

    int i = 0;
    while (i < length) {
        struct inotify_event *event = (struct inotify_event *)&buffer[i];
        char *filename = event->len ? event->name : "(none)";

        if (event->len > 0) {
            /* Tentukan tipe event berdasarkan bitmask */
            if (event->mask & IN_CREATE) {
                if (filter_mode && !valid_extension(filename))
                    goto skip; // jika genap tapi bukan .txt/.log, skip

                write_log("CREATE", filename, getpid());
                if (!filter_mode)
                    printf("[Email Dummy] File '%s' baru dibuat.\n", filename);

            } else if (event->mask & IN_MODIFY) {
                if (filter_mode && !valid_extension(filename))
                    goto skip;

                write_log("MODIFY", filename, getpid());
                if (!filter_mode)
                    printf("[Email Dummy] File '%s' telah diubah.\n", filename);

            } else if (event->mask & IN_DELETE) {
                if (filter_mode && !valid_extension(filename))
                    goto skip;

                write_log("DELETE", filename, getpid());
                if (!filter_mode)
                    printf("[Email Dummy] File '%s' telah dihapus.\n", filename);
            }
```

```
            }

        skip:
            i += sizeof(struct inotify_event) + event->len;
        }
    }

    /* Cleanup jika loop berakhir */
    close(inotify_fd);
    close(log_fd);
    return 0;
}
```

```
alanganteng@LAPTOP-CV1QG677:~$ nano file_monitor.c
alanganteng@LAPTOP-CV1QG677:~$ gcc -Wall -O2 -o file_monitor file_monitor.c
file_monitor.c: In function ‘main’:
file_monitor.c:65:5: warning: ignoring return value of ‘scanf’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
   65 |     scanf("%d", &nim_last_digit);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~
alanganteng@LAPTOP-CV1QG677:~$ ./file_monitor /path/direktori
Masukkan digit terakhir NIM Anda (0-9): 3
NIM ganjil -> mencetak notifikasi email dummy
inotify_add_watch: No such file or directory
```

Tugas 3: Custom Shell

Konsep dan Struktur yang ditunjukkan

1. main() – loop utama shell (while(1)), membaca input pengguna
2. parse_input() – memecah input menjadi token per kata
3. execute_single() – eksekusi command tanpa pipe
4. execute_single_pipe() – eksekusi 1 pipe
5. execute_multi_pipe() – eksekusi beberapa pipe
6. handle redirection() – menangani >, >>, dan <
7. execvp() – mengganti proses anak dengan program baru
8. dup2() – mengganti input/output standar.

Alur program

1. Menjalankan perintah dasar
    - Menggunakan fork() dan execvp() untuk mengeksekusi command dari user
2. Built-in Commands
    - cd <dir> - ganti direktori
    - pwd – tampilkan direkori sekarang
    - echo <teks> - cetak teks
    - exit – keluar dari shell

3. Background Process
    - Jalankan perintah di background dengan menambah & diakhir
    - Contoh: sleep 5 &.
4. Redirection (I/O)
    - > - output ke file (overwrite)
    - >> - output ke file (append)
    - < - input dari file
    - Contoh: cat < input.txt > output.txt
5. Piping (l)
    - Menghubungkan output dari satu command ke input command lain
    - NIM ganjil: bisa multiple pipe (cmd1 | cmd2 | cmd3|…..)

```c
/* Fungsi menjalankan command tanpa pipe */
void execute_single(char **args, int background) {
    pid_t pid = fork();
    if (pid == 0) {
        execvp(args[0], args);
        perror("command not found");
        exit(EXIT_FAILURE);
    } else if (pid > 0) {
        if (!background) waitpid(pid, NULL, 0);
        else printf("[Background PID %d]\n", pid);
    } else {
        perror("fork");
    }
}

/* Parsing redirection: >, >>, < */
void handle_redirection(char **args) {
    for (int i = 0; args[i]; i++) {
        if (strcmp(args[i], ">") == 0) {
            int fd = open(args[i + 1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
            dup2(fd, STDOUT_FILENO);
            close(fd);
            args[i] = NULL;
        } else if (strcmp(args[i], ">>") == 0) {
            int fd = open(args[i + 1], O_WRONLY | O_CREAT | O_APPEND, 0644);
            dup2(fd, STDOUT_FILENO);
            close(fd);
            args[i] = NULL;
        } else if (strcmp(args[i], "<") == 0) {
            int fd = open(args[i + 1], O_RDONLY);
            if (fd < 0) { perror("open input"); exit(EXIT_FAILURE); }
            dup2(fd, STDIN_FILENO);
            close(fd);
            args[i] = NULL;
        }
    }
}
```

```c
/* Eksekusi command dengan 1 pipe (ls | grep) */
void execute_single_pipe(char *cmd1[], char *cmd2[], int background) {
    int pipefd[2];
    pipe(pipefd);

    pid_t pid1 = fork();
    if (pid1 == 0) {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]); close(pipefd[1]);
        handle_redirection(cmd1);
        execvp(cmd1[0], cmd1);
        perror("exec1"); exit(EXIT_FAILURE);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
        dup2(pipefd[0], STDIN_FILENO);
        close(pipefd[1]); close(pipefd[0]);
        handle_redirection(cmd2);
        execvp(cmd2[0], cmd2);
        perror("exec2"); exit(EXIT_FAILURE);
    }

    close(pipefd[0]);
    close(pipefd[1]);

    if (!background) {
        waitpid(pid1, NULL, 0);
        waitpid(pid2, NULL, 0);
    } else printf("[Background Pipeline started]\n");
}
```

```c
/* Eksekusi multiple pipe (untuk NIM ganjil) */
void execute_multi_pipe(char ***cmds, int num_cmds, int background) {
    int pipefds[2 * (num_cmds - 1)];
    for (int i = 0; i < num_cmds - 1; i++) pipe(pipefds + i * 2);

    for (int i = 0; i < num_cmds; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            if (i > 0) dup2(pipefds[(i - 1) * 2], STDIN_FILENO);
            if (i < num_cmds - 1) dup2(pipefds[i * 2 + 1], STDOUT_FILENO);

            for (int j = 0; j < 2 * (num_cmds - 1); j++) close(pipefds[j]);

            handle_redirection(cmds[i]);
            execvp(cmds[i][0], cmds[i]);
            perror("exec multi"); exit(EXIT_FAILURE);
        }
    }

    for (int i = 0; i < 2 * (num_cmds - 1); i++) close(pipefds[i]);

    if (!background)
        for (int i = 0; i < num_cmds; i++) wait(NULL);
}
```

```c
/* Parsing input ke tokens */
void parse_input(char *input, char **args) {
    char *token = strtok(input, " \t\n");
    int i = 0;
    while (token != NULL && i < MAX_ARGS - 1) {
        args[i++] = token;
        token = strtok(NULL, " \t\n");
    }
    args[i] = NULL;
}

/* Main shell loop */
int main() {
    char input[MAX_CMD_LEN];
    printf("Masukkan digit terakhir NIM: ");
    scanf("%d", &nim_last_digit);
    getchar(); // hapus newline

    printf("\nMini Shell siap (ketik 'exit' untuk keluar)\n");

    while (1) {
        printf("myshell> ");
        if (!fgets(input, sizeof(input), stdin)) break;
        if (strcmp(input, "\n") == 0) continue;

        char input_copy[MAX_CMD_LEN];
        strcpy(input_copy, input);
        if (nim_last_digit % 2 == 0) add_history(input_copy);

        // Deteksi background
        int background = 0;
        if (input[strlen(input) - 2] == '&') {
            background = 1;
            input[strlen(input) - 2] = '\0';
        }
```

```c
    // Pisahkan command berdasarkan '|'
    char *pipe_parts[MAX_PIPES];
    int num_pipes = 0;
    char *token = strtok(input, "|");
    while (token && num_pipes < MAX_PIPES) {
        pipe_parts[num_pipes++] = token;
        token = strtok(NULL, "|");
    }

    // Built-in tanpa pipe
    if (num_pipes == 1) {
        char *args[MAX_ARGS];
        parse_input(pipe_parts[0], args);

        if (args[0] == NULL) continue;

        // Built-in commands
        if (strcmp(args[0], "exit") == 0) break;
        else if (strcmp(args[0], "cd") == 0) {
            if (args[1]) chdir(args[1]);
            else fprintf(stderr, "cd: path tidak diberikan\n");
        } else if (strcmp(args[0], "pwd") == 0) {
            char cwd[512];
            getcwd(cwd, sizeof(cwd));
            printf("%s\n", cwd);
        } else if (strcmp(args[0], "echo") == 0) {
            for (int i = 1; args[i]; i++) printf("%s ", args[i]);
            printf("\n");
        } else if (nim_last_digit % 2 == 0 && strcmp(args[0], "history") == 0) {
            show_history();
        } else {
            execute_single(args, background);
        }
    }
```

```c
        // Single pipe (genap)
        else if (num_pipes == 2 && nim_last_digit % 2 == 0) {
            char *args1[MAX_ARGS], *args2[MAX_ARGS];
            parse_input(pipe_parts[0], args1);
            parse_input(pipe_parts[1], args2);
            execute_single_pipe(args1, args2, background);
        }
        // Multiple pipes (ganjil)
        else if (num_pipes >= 2 && nim_last_digit % 2 == 1) {
            char **cmds[MAX_PIPES];
            for (int i = 0; i < num_pipes; i++) {
                cmds[i] = malloc(sizeof(char *) * MAX_ARGS);
                parse_input(pipe_parts[i], cmds[i]);
            }
            execute_multi_pipe(cmds, num_pipes, background);
            for (int i = 0; i < num_pipes; i++) free(cmds[i]);
        } else {
            fprintf(stderr, "Pipeline tidak didukung untuk variasi NIM ini.\n");
        }
    }

    printf("Shell keluar.\n");
    return 0;
}
```

```
alanganteng@LAPTOP-CV1QG677:~$ nano myshell.c
alanganteng@LAPTOP-CV1QG677:~$ gcc -Wall -O2 -o myshell myshell.c
myshell.c: In function ‘execute_single_pipe’:
myshell.c:77:5: warning: ignoring return value of ‘pipe’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
   77 |     pipe(pipefd);
      |     ^~~~~~~~~~~~
myshell.c: In function ‘execute_multi_pipe’:
myshell.c:109:44: warning: ignoring return value of ‘pipe’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  109 |     for (int i = 0; i < num_cmds - 1; i++) pipe(pipefds + i * 2);
      |                                            ^~~~~~~~~~~~~~~~~~~~~
myshell.c: In function ‘main’:
myshell.c:146:5: warning: ignoring return value of ‘scanf’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  146 |     scanf("%d", &nim_last_digit);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~
myshell.c:186:30: warning: ignoring return value of ‘chdir’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  186 |                 if (args[1]) chdir(args[1]);
      |                              ^~~~~~~~~~~~~
myshell.c:190:17: warning: ignoring return value of ‘getcwd’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  190 |                 getcwd(cwd, sizeof(cwd));
      |                 ^~~~~~~~~~~~~~~~~~~~~~~
myshell.c: In function ‘add_history’:
myshell.c:25:9: warning: ‘__builtin_strncpy’ specified bound 1024 equals destination size [-Wstringop-truncation]
   25 |         strncpy(history[MAX_HISTORY - 1], cmd, MAX_CMD_LEN);
      |         ^
alanganteng@LAPTOP-CV1QG677:~$ /myshell
-bash: /myshell: No such file or directory
alanganteng@LAPTOP-CV1QG677:~$ ./myshell
Masukkan digit terakhir NIM: 33

Mini Shell siap (ketik 'exit' untuk keluar)
myshell> exit
Shell keluar.
```

Tugas 4: System Information Tools

Konsep dan struktur yang ditunjukkan

1. read_file() – Membaca isi file menjadi string (pakai open() dan read())
2. trim() – Menghapus spasi di kiri/ kanan string
3. hex_to_ipv4() – mengubah alamat IP dari format hex ke format decimal biasa
4. tcp_state() – Mengubah kode status TCP (hex) menjadi teks seperti LISTEN atau ESTABLISHED
5. show_cpu_info() – Menampilkan model, jumlah core, dan frekuensi CPU
6. show_mem_info() – Menampilkan total, free, dan used memory.
7. show_disk_info() – Menampilkan informasi kapasitas disk
8. show_processess() – Menampilkan daftar PID, nama process, dan status
9. show_net_dev() – Menampilkan nama interface jaringan dan nama jumlah data Rx/Tx
10. show_proc_net_tcp() – khusus ganjil, Menampilkan koneksi TCP aktif
11. main() – Menjalankan semua fungsi di atas secara berurutan

Alur Program

1. CPU info (/proc/cpuinfo)
    - Menampilkan model CPU, jumlah core, dan frekuensi (Mhz).
    - Menggunakan open() dan read() untuk membaca isi file
2. Memory info (/proc/meminfo)
    - Menampilkan total, free, cached, buffer, dan memori yang digunakan
3. Disk info (statvfs("/"))
    - Mengamabil total, used dan free space pada root filesystem(/)
    - Ditampilkan dalam satuan GB
4. Proses yang sedang berjalan (/proc/<PID>/)
    - Menelusuri direktori / proc untuk mencari folder bernama angka (PID)
5. Network imterface
    - Menampilkan setiap interface jaringan
        - Bytes diterima (Rx)
        - Bytes dikirim (Tx)

6. Network sockets (NIM ganjil) - /proc/net/tcp
    - Menampilkan daftar koneksi TCP aktif di system
    - Mengkonversi alamat hexadecimal ke format IP:port

```c
/* Convert hex IPv4 in /proc/net/tcp (like "0100007F") to dotted string */
void hex_to_ipv4(const char *hex, char *out, size_t outlen) {
    unsigned long x = strtoul(hex, NULL, 16);
    unsigned int b0 = (x      ) & 0xFF;
    unsigned int b1 = (x >> 8 ) & 0xFF;
    unsigned int b2 = (x >> 16) & 0xFF;
    unsigned int b3 = (x >> 24) & 0xFF;
    snprintf(out, outlen, "%u.%u.%u.%u", b0, b1, b2, b3);
}

/* Map TCP state hex to human string */
const char *tcp_state(const char *hex) {
    int v = (int)strtol(hex, NULL, 16);
    switch (v) {
        case 1: return "ESTABLISHED";
        case 2: return "SYN_SENT";
        case 3: return "SYN_RECV";
        case 4: return "FIN_WAIT1";
        case 5: return "FIN_WAIT2";
        case 6: return "TIME_WAIT";
        case 7: return "CLOSE";
        case 8: return "CLOSE_WAIT";
        case 9: return "LAST_ACK";
        case 10: return "LISTEN";
        case 11: return "CLOSING";
        default: return "UNKNOWN";
    }
}
```

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/statvfs.h>
#include <dirent.h>
#include <ctype.h>
#include <errno.h>
#include <limits.h>
#define BUFSZ 65536

/* ---------- Helper functions ---------- */

/* Read whole file to dynamically allocated buffer. Returns NULL on error */
char *read_file(const char *path) {
    int fd = open(path, O_RDONLY);
    if (fd < 0) return NULL;
    char *buf = malloc(BUFSZ);
    if (!buf) { close(fd); return NULL; }
    ssize_t n = 0, total = 0;
    while ((n = read(fd, buf + total, BUFSZ - total - 1)) > 0) {
        total += n;
        if (total > BUFSZ - 2) break;
    }
    close(fd);
    buf[total] = '\0';
    return buf;
}

/* Trim left/right spaces */
char *trim(char *s) {
    if (!s) return s;
    while (isspace((unsigned char)*s)) s++;
    char *end = s + strlen(s) - 1;
    while (end > s && isspace((unsigned char)*end)) *end-- = '\0';
    return s;
}
```

```c
/* ---------- Sections ---------- */

void show_cpu_info(void) {
    char *s = read_file("/proc/cpuinfo");
    if (!s) {
        printf("CPU Info: cannot open /proc/cpuinfo: %s\n", strerror(errno));
        return;
    }
    char *line, *saveptr;
    char *model = NULL;
    int core_count = 0;
    double mhz = 0.0;

    for (line = strtok_r(s, "\n", &saveptr); line; line = strtok_r(NULL, "\n", &saveptr)) {
        char *p = strchr(line, ':');
        if (!p) {
            if (strncmp(line, "processor", 9) == 0) core_count++;
            continue;
        }
        char key[512], val[512];
        strncpy(key, line, p - line);
        key[p - line] = '\0';
        strcpy(val, p + 1);
        trim(key); trim(val);

        if (!model && (strcmp(key, "model name") == 0 || strcmp(key, "Processor") == 0)) {
            model = strdup(val);
        }
        if (mhz == 0.0 && strcmp(key, "cpu MHz") == 0) {
            mhz = atof(val);
        }
        if (strcmp(key, "processor") == 0) {
            core_count++; /* fallback if not counted earlier */
        }
    }
```

```c
    printf("=== CPU Info ===\n");
    if (model) printf("Model: %s\n", model);
    else printf("Model: (not found)\n");
    if (core_count > 0) printf("Cores (counted): %d\n", core_count);
    else printf("Cores: (not found)\n");
    if (mhz > 0.0) printf("Frequency (MHz): %.2f\n", mhz);
    else printf("Frequency: (not found)\n");
    printf("\n");

    free(model);
    free(s);
}

void show_mem_info(void) {
    char *s = read_file("/proc/meminfo");
    if (!s) {
        printf("Memory Info: cannot open /proc/meminfo: %s\n", strerror(errno));
        return;
    }
    char *line, *saveptr;
    unsigned long MemTotal=0, MemFree=0, Buffers=0, Cached=0, SReclaimable=0, Shmem=0;

    for (line = strtok_r(s, "\n", &saveptr); line; line = strtok_r(NULL, "\n", &saveptr)) {
        char key[512];
        unsigned long val = 0;
        if (sscanf(line, "%127s %lu", key, &val) < 2) continue;
        if (strcmp(key, "MemTotal:") == 0) MemTotal = val;
        else if (strcmp(key, "MemFree:") == 0) MemFree = val;
        else if (strcmp(key, "Buffers:") == 0) Buffers = val;
        else if (strcmp(key, "Cached:") == 0) Cached = val;
        else if (strcmp(key, "SReclaimable:") == 0) SReclaimable = val;
        else if (strcmp(key, "Shmem:") == 0) Shmem = val;
    }
```

```
    unsigned long used_kb = 0;
    if (MemTotal) {
        unsigned long cached_total = Cached + SReclaimable;
        if (MemTotal > MemFree + Buffers + cached_total) used_kb = MemTotal - MemFree - Buffers - cached_total;
        else used_kb = MemTotal - MemFree; /* fallback */
    }

    printf("=== Memory Info (kB) ===\n");
    printf("MemTotal: %lu kB\n", MemTotal);
    printf("MemFree : %lu kB\n", MemFree);
    printf("Buffers : %lu kB\n", Buffers);
    printf("Cached  : %lu kB\n", Cached);
    printf("SReclaimable: %lu kB\n", SReclaimable);
    printf("Shmem   : %lu kB\n", Shmem);
    printf("Used (approx): %lu kB\n\n", used_kb);

    free(s);
}

void show_disk_info(void) {
    struct statvfs st;
    if (statvfs("/", &st) != 0) {
        printf("Disk Info: statvfs('/') failed: %s\n\n", strerror(errno));
        return;
    }
    unsigned long long total = (unsigned long long)st.f_blocks * st.f_frsize;
    unsigned long long free  = (unsigned long long)st.f_bfree  * st.f_frsize;
    unsigned long long avail = (unsigned long long)st.f_bavail  * st.f_frsize;
    unsigned long long used  = total - free;

    printf("=== Disk Info (root filesystem) ===\n");
    printf("Total: %.2f GB\n", total / 1024.0 / 1024.0 / 1024.0);
    printf("Used : %.2f GB\n", used  / 1024.0 / 1024.0 / 1024.0);
    printf("Free : %.2f GB (available: %.2f GB)\n\n",
           free / 1024.0 / 1024.0 / 1024.0,
           avail / 1024.0 / 1024.0 / 1024.0);
}
```

```
void show_processes(void) {
    DIR *d = opendir("/proc");
    if (!d) {
        printf("Processes: cannot open /proc: %s\n\n", strerror(errno));
        return;
    }
    struct dirent *ent;
    printf("=== Running processes (PID, NAME, STATE) ===\n");
    while ((ent = readdir(d)) != NULL) {
        const char *name = ent->d_name;
        int isnum = 1;
        for (const char *p = name; *p; ++p) if (!isdigit((unsigned char)*p)) { isnum = 0; break; }
        if (!isnum) continue;
        char commpath[512], statuspath[512];
        snprintf(commpath, sizeof(commpath), "/proc/%s/comm", name);
        snprintf(statuspath, sizeof(statuspath), "/proc/%s/status", name);

        FILE *fc = fopen(commpath, "r");
        char comm[256] = {0};
        if (fc) {
            if (fgets(comm, sizeof(comm), fc)) trim(comm);
            fclose(fc);
        } else {
            strcpy(comm, "(no comm)");
        }
```

```
        FILE *fs = fopen(statuspath, "r");
        char state[128] = "(unknown)";
        if (fs) {
            char line[512];
            while (fgets(line, sizeof(line), fs)) {
                if (strncmp(line, "State:", 6) == 0) {
                    char *p = strchr(line, ':');
                    if (p) {
                        strcpy(state, trim(p+1));
                    }
                    break;
                }
            }
            fclose(fs);
        }

        printf("%5s  %-25s  %s\n", name, comm, state);
    }
    closedir(d);
    printf("\n");
}
```

```
/* Network interfaces: parse /proc/net/dev */
void show_net_dev(void) {
    char *s = read_file("/proc/net/dev");
    if (!s) {
        printf("Net Dev: cannot open /proc/net/dev: %s\n\n", strerror(errno));
        return;
    }
    printf("=== Network Interfaces (/proc/net/dev) ===\n");
    char *line, *saveptr;
    int lineno = 0;
    for (line = strtok_r(s, "\n", &saveptr); line; line = strtok_r(NULL, "\n", &saveptr)) {
        lineno++;
        if (lineno <= 2) continue; /* skip headers */
        char *p = strchr(line, ':');
        if (!p) continue;
        char ifname[64];
        strncpy(ifname, line, p - line);
        ifname[p - line] = '\0';
        trim(ifname);
        unsigned long long rx_bytes=0, tx_bytes=0;
        /* after colon, fields: rx_bytes ... tx_bytes are later field 9? We'll sscanf accordingly */
        char *rest = p + 1;
```

```
        /* Use sscanf to pull first & ninth numeric fields: rx_bytes is first, tx_bytes is 9th */
        unsigned long long fields[16] = {0};
        int i = 0;
        char *tok = strtok_r(rest, " \t", &saveptr);
        while (tok && i < 16) {
            if (strlen(tok) > 0) {
                fields[i++] = strtoull(tok, NULL, 10);
            }
            tok = strtok_r(NULL, " \t", &saveptr);
        }
        if (i >= 9) {
            rx_bytes = fields[0];
            tx_bytes = fields[8];
        }
        printf("%-10s  Rx: %llu bytes  Tx: %llu bytes\n", ifname, rx_bytes, tx_bytes);
    }
    printf("\n");
    free(s);
}

/* Parse /proc/net/tcp and print local/remote addresses + state (additional for NIM ganjil) */
void show_proc_net_tcp(void) {
    char *s = read_file("/proc/net/tcp");
    if (!s) {
        printf("Net TCP: cannot open /proc/net/tcp: %s\n\n", strerror(errno));
        return;
    }
    printf("=== /proc/net/tcp (sockets) ===\n");
    char *line, *saveptr;
    int lineno = 0;
    for (line = strtok_r(s, "\n", &saveptr); line; line = strtok_r(NULL, "\n", &saveptr)) {
        lineno++;
        if (lineno == 1) continue; /* skip header */
```

```
        /* We'll parse by tokens */
        char *cols[20];
        int col = 0;
        char *p = line;
        while (col < 20 && (cols[col] = strsep(&p, " \t")) != NULL) {
            if (strlen(cols[col]) == 0) continue;
            col++;
        }
        if (col < 4) continue;
        /* cols[1] might be of form "0100007F:0035" (local), cols[2] rem, cols[3] state */
        char *local = cols[1];
        char *rem   = cols[2];
        char *state = cols[3];

        char loc_ip_hex[64] = {0}, loc_port_hex[32] = {0};
        char rem_ip_hex[64] = {0}, rem_port_hex[32] = {0};

        char *pcol = strchr(local, ':');
        if (pcol) {
            strncpy(loc_ip_hex, local, pcol - local);
            loc_ip_hex[pcol - local] = '\0';
            strcpy(loc_port_hex, pcol + 1);
        }
        pcol = strchr(rem, ':');
        if (pcol) {
            strncpy(rem_ip_hex, rem, pcol - rem);
            rem_ip_hex[pcol - rem] = '\0';
            strcpy(rem_port_hex, pcol + 1);
        }

        char loc_ip[64] = {0}, rem_ip[64] = {0};
        hex_to_ipv4(loc_ip_hex, loc_ip, sizeof(loc_ip));
        hex_to_ipv4(rem_ip_hex, rem_ip, sizeof(rem_ip));
        int loc_port = (int)strtol(loc_port_hex, NULL, 16);
        int rem_port = (int)strtol(rem_port_hex, NULL, 16);
```

```
        printf("Local: %s:%d  -> Remote: %s:%d  State: %s\n",
               loc_ip, loc_port, rem_ip, rem_port, tcp_state(state));
    }
    printf("\n");
    free(s);
}

/* Optional: show loadavg if needed (not for "ganjil" but useful) */
void show_loadavg(void) {
    char *s = read_file("/proc/loadavg");
    if (!s) {
        printf("Loadavg: cannot open /proc/loadavg: %s\n\n", strerror(errno));
        return;
    }
    /* format: "0.00 0.01 0.05 1/123 4567" */
    char a[64], b[64], c[64];
    if (sscanf(s, "%63s %63s %63s", a, b, c) >= 3) {
        printf("Load average (1,5,15 min) : %s, %s, %s\n\n", a, b, c);
    }
    free(s);
}

/* ---------- Main ---------- */

int main(void) {
    printf("\nSystem Information Tool - versi NIM GANJIL\n\n");

    show_cpu_info();
    show_mem_info();
    show_disk_info();
    show_processes();
    show_net_dev();

    /* NIM ganjil: tambahkan network socket info dari /proc/net/tcp */
    show_proc_net_tcp();
```

```
    /* (bila ingin) juga bisa menampilkan loadavg - tapi ini untuk genap. Uncomment jika mau:
     * show_loadavg();
     */

    return 0;
}
```

```
alanganteng@LAPTOP-CV1QG677:~$ nano sysinfo_odd.c
alanganteng@LAPTOP-CV1QG677:~$ gcc -O2 -Wall sysinfo_odd.c -o sysinfo_odd
alanganteng@LAPTOP-CV1QG677:~$ ./sysinfo_odd

System Information Tool - versi NIM GANJIL

*** buffer overflow detected ***: terminated
Aborted (core dumped)
alanganteng@LAPTOP-CV1QG677:~$
```